

UNIT – II

SEARCHING FOR SOLUTIONS IN AI

Meaning of Search

In Artificial Intelligence, **search** refers to the systematic exploration of possible states to find a sequence of actions that transforms an **initial state** into a **goal state**.

Searching is fundamental because:

- AI agents often do not know the solution in advance
 - They must explore alternatives intelligently
-

Components of a Search Problem

A problem in AI can be formally defined using:

1. **Initial State**
The starting point of the agent.
 2. **State Space**
The set of all possible states reachable from the initial state.
 3. **Actions / Operators**
Rules that move the agent from one state to another.
 4. **Transition Model**
Describes the result of performing an action.
 5. **Goal Test**
Checks whether a given state satisfies the goal.
 6. **Path Cost**
Numerical cost of a path (distance, time, energy, etc.).
-

Classification of Search Algorithms

Search algorithms are broadly classified into:

1. **Uninformed (Blind) Search**
2. **Informed (Heuristic) Search**
3. **Adversarial Search (Game Playing)**

UNINFORMED SEARCH STRATEGIES

Uninformed search strategies **do not use any heuristic or additional domain knowledge.**

They only know:

- Where they start
- What actions are possible
- Whether the goal is reached

a) BREADTH FIRST SEARCH (BFS)

Definition

Breadth First Search is an uninformed search algorithm that explores the state space **level by level**, expanding all nodes at one depth before moving to the next depth.

Working Principle

- BFS starts from the root (initial state)
- It explores all immediate neighbors first
- Then neighbors of neighbors, and so on

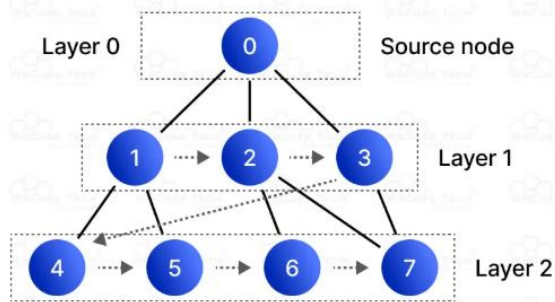
Data Structure Used

- **Queue (First In First Out – FIFO)**

Algorithm Steps

1. Insert the initial state into a queue.
2. Remove the first node from the queue.
3. Check if it is the goal.
4. If not, expand it and insert all its children at the end of the queue.
5. Repeat until the goal is found or the queue becomes empty.

Breadth-First Search Algorithm



Example

Finding the **shortest path** between two cities when all roads have equal distance.

Time Complexity

$$O(b^d)$$

where

- b = branching factor
- d = depth of the shallowest goal node

Space Complexity

$$O(b^d)$$

Requires storing all nodes at each level.

Applications

- GPS navigation (equal-cost paths)

Drawbacks

- Very high memory requirement
- Not suitable for large or infinite state spaces

DEPTH FIRST SEARCH (DFS)

Definition

Depth First Search is an uninformed search algorithm that explores **as deep as possible along a branch before backtracking**.

Working Principle

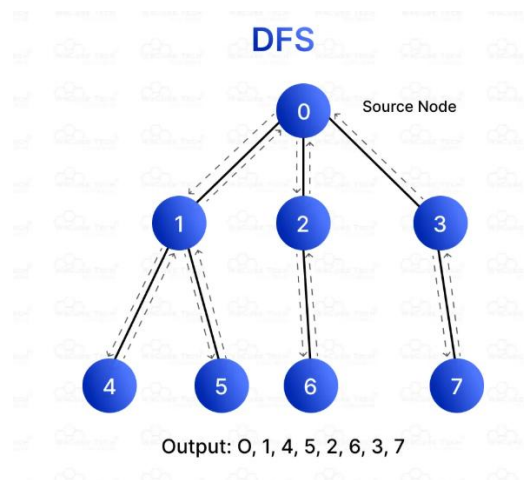
- DFS selects one child and keeps expanding it
- When a dead end is reached, it backtracks

Data Structure Used

- **Stack (LIFO)** or recursion

Algorithm Steps

1. Push the initial node onto the stack.
2. Pop the top node.
3. Check for goal.
4. If not goal, push its children onto the stack.
5. Repeat until stack is empty or goal is found.



Example

Maze exploration where one path is fully explored before trying alternatives.

Time Complexity

$$O(b^m)$$

where m is the maximum depth of the search tree.

Space Complexity

$$O(bm)$$

Much lower than BFS.

Applications

- Puzzle solving
- Cycle detection

Drawbacks

- Can get stuck in infinite paths
- May miss the solution
- Does not guarantee shortest path

HEURISTIC (INFORMED) SEARCH

Meaning

Heuristic search uses **additional knowledge (heuristics)** to guide the search toward the goal more efficiently.

Heuristic Function

$h(n)$ = estimated cost from node n to the goal

Hill Climbing

Hill Climbing is a **heuristic search algorithm** used in Artificial Intelligence to solve **optimization problems**. It belongs to the category of **local search algorithms**, where the search starts from an initial state and continuously improves the solution by making small local changes.

The algorithm works on the idea of **incremental improvement**, where each new state is evaluated using a heuristic or evaluation function, and the algorithm moves only if a better state is found.

Steps Involved in Hill Climbing Algorithm

The Hill Climbing algorithm follows these steps:

1. Initial State

The algorithm starts with an arbitrary or randomly chosen initial state.

2. **Generate Neighbors**

All possible neighboring states are generated by making small changes to the current state.

3. **Evaluation**

Each neighboring state is evaluated using an **evaluation (heuristic) function**.

4. **Move to Better State**

If a neighbor has a better evaluation value than the current state, the algorithm moves to that neighbor.

5. **Termination**

The algorithm stops when no neighboring state offers a better value. At this point, a solution is reached.

Features of Hill Climbing Algorithm

1. Heuristic-Based

Hill Climbing uses a heuristic or evaluation function to decide which state is better.

2. Greedy Approach

It follows a **greedy strategy**, meaning it always chooses the best immediate option without considering future consequences.

3. Local Search

The algorithm focuses only on the current state and its neighbors rather than exploring the entire search space.

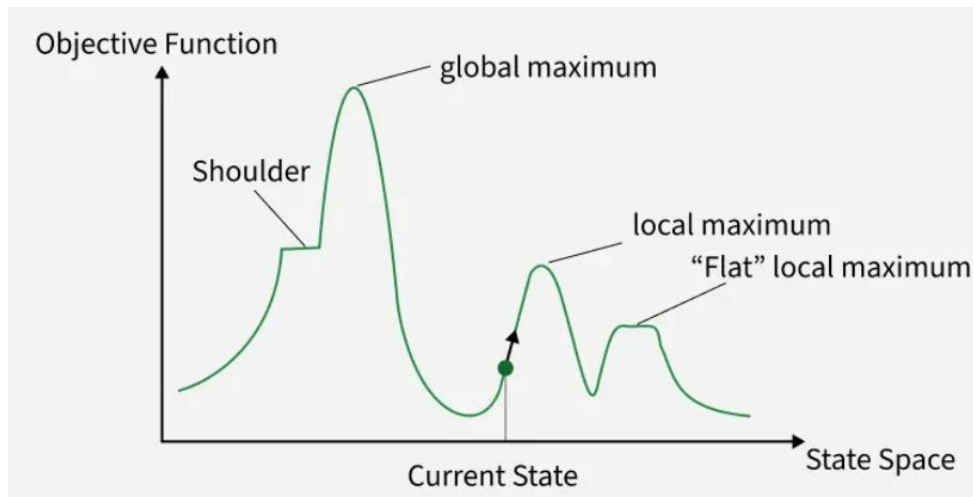
4. Iterative Improvement

The solution is gradually improved step by step.

State-Space Diagram in Hill Climbing

The **state-space diagram** represents all possible states and their evaluation values.

- **X-axis:** Possible states
- **Y-axis:** Value of the objective function



Important Regions in the Diagram

- **Local Maximum:** A state better than its neighbors but not the best overall
- **Global Maximum:** The best possible solution
- **Plateau:** Flat region where neighboring states have equal values
- **Ridge:** A narrow peak that can mislead the algorithm
- **Shoulder:** A flat region with a possible upward path

Problems in Hill Climbing

Hill Climbing suffers from several limitations:

1. Local Maximum Problem

The algorithm may stop at a local maximum instead of reaching the global maximum.

2. Plateau Problem

When all neighboring states have the same value, the algorithm cannot decide where to move.

3. Ridge Problem

The algorithm may oscillate or stop prematurely due to narrow paths.

Because of these issues, Hill Climbing does not always guarantee the optimal solution.

Advantages of Hill Climbing

- Simple and easy to understand
- Requires less memory
- Fast and efficient for large search spaces

- Useful when an approximate solution is acceptable

Disadvantages of Hill Climbing

- Can get stuck in local maxima
- No guarantee of global optimal solution
- Depends heavily on the initial state
- Not suitable for complex or multi-modal problems

A* Algorithm

The A* (A-star) algorithm is a **heuristic search algorithm** used to find the **most optimal path** between a start state and a goal state. It is one of the most powerful and widely used search techniques in Artificial Intelligence.

The algorithm was developed in **1968 by Peter Hart, Nils Nilsson, and Bertram Raphael** as an improvement over **Dijkstra's algorithm**.

A* is called an **informed search algorithm** because it uses **heuristic information** to guide the search toward the goal efficiently.

Key Cost Functions Used in A*

A* uses three important cost functions:

1. $g(n)$ – Path Cost So Far

- Represents the **actual cost** of reaching node n from the start node.
- It is the accumulated cost of all edges along the path so far.

2. $h(n)$ – Heuristic Cost

- Represents the **estimated cost** to reach the goal from node n .
- It is an **approximation**, not an exact value.
- Provided by a heuristic function.

3. $f(n)$ – Total Estimated Cost

$$f(n) = g(n) + h(n)$$

Represents the **estimated total cost** of the best path through node n .

- A* always expands the node with the **lowest $f(n)$** value.

Working of A* Algorithm

The working of the A* algorithm can be explained step by step:

1. Initialization

The start node is placed in an *open list* (nodes to be explored).
Its $f(n)$ value is calculated.

2. Node Selection

The algorithm selects the node with the **lowest $f(n)$** value from the open list.

3. Goal Test

If the selected node is the goal node, the algorithm stops and returns the solution path.

4. Node Expansion

The selected node is expanded, and its neighboring nodes are evaluated.

5. Cost Update

For each neighbor, $g(n)$, $h(n)$, and $f(n)$ values are calculated.
If a better path is found, values are updated.

6. Repeat

The process continues until the goal is reached or no nodes remain.

Heuristic Function in A*

The **heuristic function $h(n)$** plays a crucial role in the performance of A*. It estimates the remaining distance from the current node to the goal.

A good heuristic helps the algorithm:

- Reach the goal faster
- Avoid unnecessary node expansions
- Maintain optimality

Properties of a Good Heuristic

1. Admissibility

- A heuristic is **admissible** if it never overestimates the true cost to reach the goal.
- This guarantees that A* always finds the **optimal path**.

Example:

Straight-line distance in maps is admissible.

2. Consistency (Monotonicity)

- A heuristic is consistent if the estimated cost from the current node is always less than or equal to:
 - The cost of moving to a neighbor + heuristic from that neighbor
- This property avoids re-expansion of nodes and improves efficiency.

Common Heuristics Used in A*

1. Manhattan Distance

- Used in grid environments
- Movement allowed only horizontally and vertically

2. Euclidean Distance

- Straight-line distance
- Used when movement is allowed in any direction

3. Chebyshev Distance

- Used when diagonal movement is allowed

Example of A* Algorithm (Conceptual)

Consider a **map navigation problem**:

- **Initial State:** Current location
- **Goal State:** Destination
- **$g(n)$:** Distance already traveled
- **$h(n)$:** Estimated remaining distance
- **$f(n)$:** Total estimated distance

A* selects the route with the **lowest total estimated cost**, ensuring the shortest and most efficient path.

Applications of A* Algorithm

A* is widely used in real-world AI applications such as:

- **Game pathfinding** (NPC movement)
- **Robotics navigation**
- **GPS and map navigation systems**

- **AI planning and decision making**

Advantages of A* Algorithm

- **Optimal:** Finds the shortest path with an admissible heuristic
- **Complete:** Always finds a solution if one exists
- **Efficient:** Explores fewer nodes than uninformed searches
- **Flexible:** Can be adapted to different problem domains
- **Widely applicable:** Works with weighted and unweighted graphs

Limitations of A* Algorithm

- **High memory usage** due to storing many nodes
- **Heuristic dependent:** Poor heuristic reduces efficiency
- **Computational overhead** in large or dense graphs
- Not ideal for **memory-constrained or real-time systems**

AO* Algorithm

The **AO*** (AND–OR Star) algorithm is an **advanced heuristic search algorithm** used in Artificial Intelligence for **problem solving and decision making**. It is an **extension of the A*** algorithm and is especially suitable for problems that can be decomposed into **interdependent subproblems**.

Unlike A*, which searches along a **single best path**, AO* can evaluate **multiple paths simultaneously** using an **AND–OR graph representation**.

AND–OR Graph Representation

AO* operates on a **directed acyclic graph (DAG)** known as an **AND–OR graph**.

Node Types

1. OR Nodes

- Represent **alternative choices**
- Solving **any one child** is sufficient
- Cost is the **minimum** of child costs

2. AND Nodes

- Represent **mandatory subproblems**
- **All child nodes must be solved**
- Cost is the **maximum** of child costs

This structure allows AO* to model **problem reduction**, where a main problem is reduced into smaller subproblems.

Heuristic Function in AO*

AO* uses a heuristic function similar to A*:

$$h(n) = \text{estimated cost to reach the goal from node } n$$

Heuristics guide the search toward promising paths and reduce unnecessary exploration.

Cost Computation Rules

Cost of OR Node

$$C(n) = \min\{C(c_1), C(c_2), \dots, C(c_k)\}$$

Cost of AND Node

$$C(n) = \max\{C(c_1), C(c_2), \dots, C(c_k)\}$$

Total Estimated Cost

$$f(n) = C(n) + h(n)$$

AO* always expands the node or path with the **lowest f(n)** value.

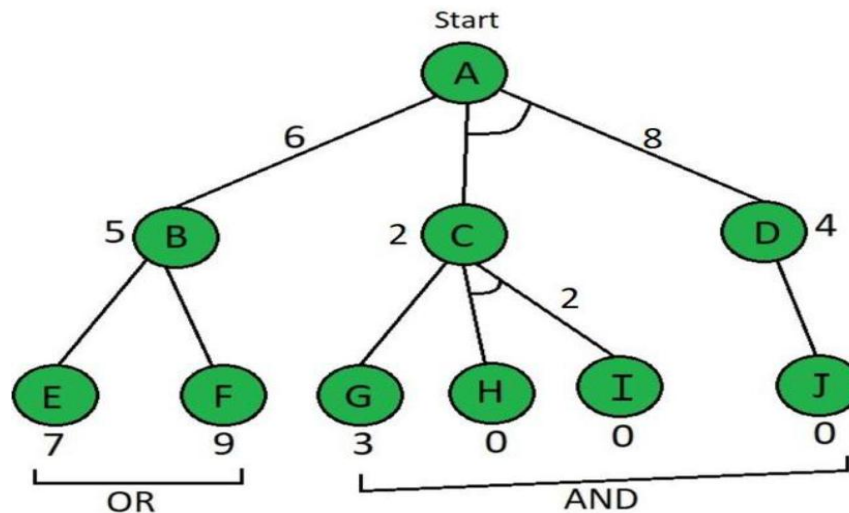
Working Principle of AO* Algorithm

1. Start from the **initial node**
2. Identify whether the node is **AND or OR**
3. Compute costs using heuristics
4. Select the **least-cost solution graph**
5. Expand nodes recursively
6. Update heuristic values from **leaf nodes to root**
7. Stop when the **solution graph is completely solved**

Problem Description

Each node has:

- A heuristic value $h(n)$
- Each edge has cost = 1



The objective is to find the **least-cost solution graph** from root node A to goal nodes.

Step 1: Initial Evaluation from Root Node A

OR Choice:

$$f(A \rightarrow B) = g(B) + h(B) = 1 + 5 = 6$$

AND Choice:

$$f(A \rightarrow C + D) = (1 + 2) + (1 + 4) = 8$$

Decision:

Since $6 < 8$, path $A \rightarrow B$ is selected.

Step 2: Expansion of Node B

Evaluate children of B:

$$f(B \rightarrow E) = 1 + 7 = 8$$

$$f(B \rightarrow F) = 1 + 9 = 10$$

Decision:

Choose $B \rightarrow E$ (minimum cost = 8)

Since B's heuristic changes, **update heuristic of A:**

$$f(A \rightarrow B) = 1 + 8 = 9$$

Step 3: Re-evaluation at Node A

Compare updated costs:

- $f(A \rightarrow B) = 9$
- $f(A \rightarrow C + D) = 8$

Decision:

Now $A \rightarrow C + D$ is selected.

Step 4: Expansion of Node C

$$f(C \rightarrow G) = 1 + 3 = 4$$

$$f(C \rightarrow H + I) = (1 + 0) + (1 + 0) = 2$$

Decision:

Choose $C \rightarrow H + I$

Since H and I are goal nodes with heuristic 0, they are **solved**.

Step 5: Expansion of Node D

$$f(D \rightarrow J) = 1 + 0 = 1$$

Now update parent:

$$f(A \rightarrow C + D) = (1 + 2) + (1 + 1) = 5$$

Final Decision

Since $A \rightarrow C + D$ has the **lowest cost (5)**, it becomes the **final solved solution graph**.

Adversarial Search

Adversarial search is used in **competitive environments** where two or more agents have **conflicting goals**. It is most commonly applied to **two-player, zero-sum games**, where one player's gain is exactly equal to the other player's loss. In such scenarios, no win-win outcome is possible.

The **Minimax algorithm** is a fundamental adversarial search technique that helps an AI agent make **optimal decisions** by considering both its own moves and the **best possible counter-moves of the opponent**.

Minimax Algorithm?

The Minimax algorithm is a **recursive decision-making strategy** that operates on a **game tree**. The objective is to determine the **best possible move** for a player assuming that **both players play optimally**.

- One player is called **MAX (Maximizer)** – tries to **maximize** the score.
- The other player is called **MIN (Minimizer)** – tries to **minimize** the maximizer's score.

The algorithm evaluates all possible future game states and chooses the move that leads to the **most favorable guaranteed outcome**.

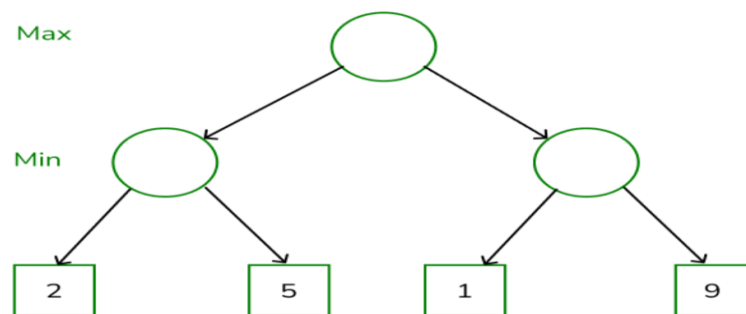
Key Terminologies

1. **Game Tree**
A tree structure representing all possible moves and game states from the current position.
2. **Initial State**
The current configuration of the game board.
3. **Terminal State**
The final game states where the game ends (win, loss, or draw).
4. **MAX Node**
A node where the player tries to select the **maximum value**.
5. **MIN Node**
A node where the opponent tries to select the **minimum value**.
6. **Heuristic Evaluation Function**
A function that assigns numerical values to non-terminal game states based on factors like board control, material advantage, or position strength.

Working Principle of Minimax Algorithm

1. The algorithm starts at the **root node**, which represents the current game state.
 2. It expands the game tree downwards until it reaches **terminal nodes**.
 3. Each terminal node is assigned a **utility value**.
 4. These values are then **propagated upwards**:
 - MIN nodes select the **minimum** value from their children.
 - MAX nodes select the **maximum** value from their children.
 5. The root node finally selects the move associated with the **optimal value**.
-

Explanation of the Given Example (Binary Game Tree)



Step 1: Maximizer's Initial Decision

- The root node belongs to the **Maximizer**.
- It has two choices: **left subtree** or **right subtree**.

Step 2: Minimizer's Choice (Left Subtree)

- On moving left, the Maximizer reaches a **Minimizer node**.
- The Minimizer evaluates two leaf values: **2 and 5**.
- Since the Minimizer aims to reduce MAX's score, it chooses **2**.
- This value is passed back to the root.

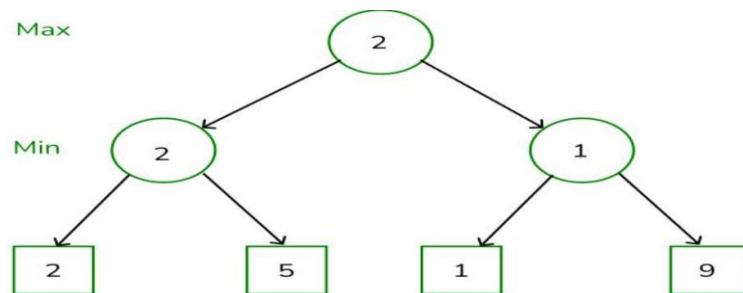
Step 3: Exploring the Right Subtree

- The Maximizer now explores the right subtree.

- Again, it encounters a Minimizer node.
- The Minimizer compares its available values and selects **1** (minimum).
- This value is also passed back to the root.

Step 4: Final Decision

- The Maximizer compares the two received values: **2 (left)** and **1 (right)**.
- Since MAX prefers the highest guaranteed outcome, it selects **2**.
- Hence, the optimal value for the game is **2**, and the left path is chosen.



Final outcome of binary game tree

Optimal Strategy in Minimax

- Terminal states provide direct utility values.
 - Non-terminal states derive their values from child nodes.
 - The algorithm ensures that the Maximizer selects a move that yields the **best possible worst-case outcome**.
-

Time and Space Complexity

Time Complexity

$$O(b^m)$$

Where:

- **b** = branching factor (number of legal moves)
- **m** = maximum depth of the game tree

Space Complexity

- Worst case: $O(bm)$

- If nodes are generated one at a time: $O(m)$

Drawbacks of Minimax Algorithm

1. Computationally expensive for large game trees
 2. Inefficient for games with high branching factors
 3. Not practical without optimization techniques
 4. Requires full game tree exploration
-

Applications

- Chess
- Tic-Tac-Toe
- Checkers
- Turn-based strategy games

Alpha–Beta Pruning

Alpha–Beta pruning is an **optimization technique** for the Minimax algorithm. It improves efficiency by **eliminating branches** of the game tree that cannot influence the final decision.

The result obtained using Alpha–Beta pruning is **exactly the same as Minimax**, but with **significantly fewer node evaluations**.

Basic Idea of Alpha–Beta Pruning

The algorithm uses two parameters:

- **Alpha (α):**
The best (highest) value that the Maximizer can guarantee so far.
- **Beta (β):**
The best (lowest) value that the Minimizer can guarantee so far.

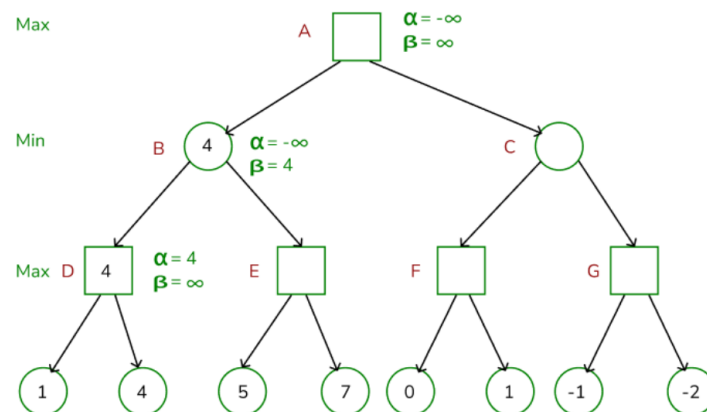
Pruning Condition

If at any point:

$$\alpha \geq \beta$$

then further exploration of that branch is **stopped (pruned)**.

Working of Alpha–Beta Pruning



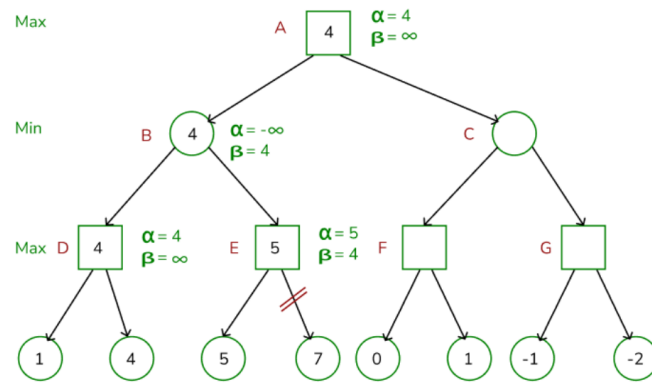
Initial Setup

- Root node (Maximizer) starts with:
 - $\alpha = -\infty$
 - $\beta = +\infty$
-

Step 1: Exploring Left Subtree

- Root selects node **B**.
 - Node B explores child **D**.
 - D returns a value **4**.
 - This updates:
 - α at D
 - β at B becomes **4**
-

Step 2: Pruning at Node E



- Node B explores child **E**.
- E evaluates its left leaf and gets **5**.
- Alpha at E becomes **5**.
- Since:

$$\alpha(5) \geq \beta(4)$$

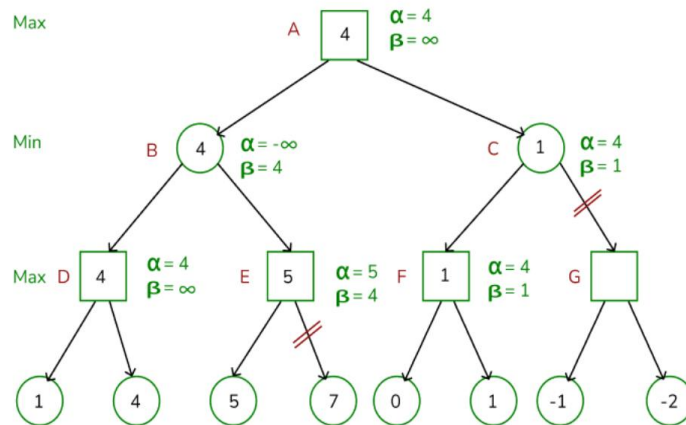
- The remaining child of E is **pruned**.
- No need to explore it further.

Step 3: Updating Root Node

- B returns value **4**.
- Root updates its α to **4**.

Step 4: Exploring Right Subtree

- Root now explores node **C**.
- C explores node **F**.
- F returns values **0 and 1**, resulting in **1**.
- C updates β to **1**.



Step 5: Pruning at Node C

- At node C:

$$\alpha(4) \geq \beta(1)$$

- Entire subtree of node **G** is pruned.
- This is because the Maximizer already has a better option (value 4).

Final Outcome

- Root node chooses the left subtree.
- Optimal value remains **4**.
- Several branches are skipped without affecting the final result.

Time and Space Complexity

Time Complexity

- Best case:

$$O(b^{m/2})$$

- Worst case (same as Minimax):

$$O(b^m)$$

Space Complexity

- Similar to Minimax: $O(m)$

Advantages of Alpha–Beta Pruning

1. Faster than Minimax
2. Fewer node evaluations
3. Enables deeper look-ahead
4. Suitable for complex games

Limitations

1. Effectiveness depends on node ordering
2. Still exponential in worst case
3. Requires heuristic evaluation functions

Applications

- Chess engines
- Competitive board games
- Strategic AI systems